

Aegis — A Hybrid ML + LLM Credential Defense Layer for LLM Agents

Gauntlet AI Capstone Proposal

The Problem

LLM agents must keep real credentials (API keys, database passwords, OAuth tokens) in the same context window as untrusted content to function. This creates a structural attack surface: indirect prompt injection can cause credential exfiltration. Text-level defenses fail against encodings, slow multi-turn leakage, and — most critically — tool-call arguments. Recent benchmarks show frontier agents achieve only 17% verified success on related tasks, with up to 40% of functionally passing patches rejected for cryptographic or semantic violations. The problem is architectural, not incremental.

Project Direction

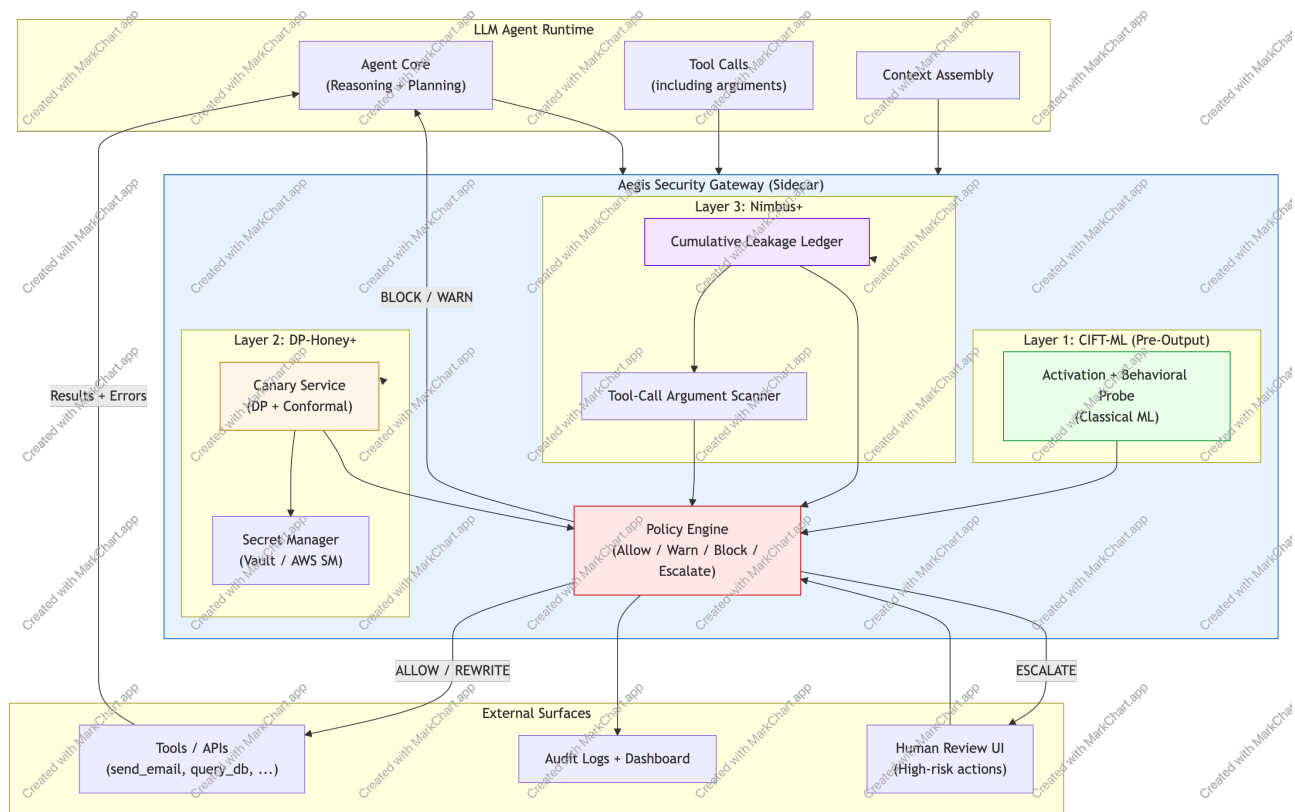
This is a **Combine ML with LLM applications** project (Direction A). Classical machine learning provides fast, cheap prediction of exfiltration risk from activation features, provenance signals, and behavioral traces. LLM agents handle proposal, repair, and runtime decision-making. Neither paradigm alone solves the problem.

Technical Approach

Aegis is a runtime security gateway with three cooperating layers that treat AI as a first-class primitive:

- **CIFT-ML**: A classical ML probe (trained on activation and behavioral features) predicts credential access *before* any token is generated.
- **DP-Honey+**: A calibrated canary service that plants statistically realistic honeytokens and integrates with secret managers so real credentials are never visible to the model.
- **Nimbus+**: A cumulative leakage accountant that tracks risk across turns and sessions, with explicit tool-call argument scanning.

The three layers feed a policy engine. The ML components improve over time from agent trajectories and verifier outcomes, creating a closed loop.



Scope (Two Weeks)

Minimum shippable: A working sidecar that intercepts context and tool calls, runs the three layers, and demonstrates measurable improvement over text-only baselines on held-out encoded + multi-turn cases.

Ambitious version: Closed-loop retraining of the ML probe from agent outcomes + first quantitative results on tool-call argument exfiltration.

Why It's Compelling

This directly attacks a documented, high-stakes gap that no existing product or research system fully closes. It is visibly ambitious (hybrid closed-loop system on a structural problem), measurable, and timely. Team members will own distinct, high-visibility layers with clear ownership and strong demo potential.

Hard Parts / Risks

- Tool-call argument scanning has almost no published precedent.
- Pre-output detection requires white-box access or strong black-box proxies.
- False-positive rate must stay low enough for real use; over-blocking kills utility.
- The ML components must demonstrably improve the LLM pipeline, not just add overhead.

Demo Sketch (10 minutes)

Live side-by-side comparison: baseline agent vs. Aegis-protected agent under three attacks (encoded single-turn, low-rate multi-turn dripping, tool-call argument injection). Real-time visualization of the three layers firing, with quantitative results table (detection rate, false blocks, latency) shown live. End with a short architecture diagram and the closed-loop retraining story.